

Funzioni in Python

Funzioni in Python

Come creare blocchi di codice riutilizzabili per non ripetere sempre le stesse cose.

Perché usare le funzioni?

Immagina di dover calcolare la media dei voti tre volte in un programma. Senza funzioni, dovresti scrivere la stessa formula tre volte. Se poi vuoi cambiare come si calcola la media, devi ricordarti di cambiarla in tutti e tre i posti.

Con le funzioni, scrivi il calcolo **una volta sola**, gli dai un nome, e lo riusi quante volte vuoi. Se cambia qualcosa, modifichi solo la funzione.

Creare una funzione

Si usa la parola chiave `def` (abbreviazione di "define", definire):

```
def saluta():  
    print("Ciao!")
```

Questa funzione si chiama `saluta` e, quando viene eseguita, stampa "Ciao!". Ma attenzione: definire la funzione non la esegue ancora.

Chiamare una funzione

Per eseguire la funzione, la "chiami" scrivendone il nome seguito dalle parentesi:

```
saluta() # esegue la funzione → stampa "Ciao!"  
saluta() # puoi richiamarla quante volte vuoi  
saluta()
```

Passare informazioni: i parametri

Le funzioni diventano molto più utili quando possono ricevere informazioni dall'esterno. Queste informazioni si chiamano **parametri**:

```
def saluta(nome):  
    print(f"Ciao, {nome}!")  
  
saluta("Alice")    # Ciao, Alice!  
saluta("Bob")      # Ciao, Bob!  
saluta("Carlo")    # Ciao, Carlo!
```

La stessa funzione si comporta in modo diverso in base a cosa le passi. Puoi avere più parametri:

```
def somma(a, b):  
    print(a + b)  
  
somma(3, 5)        # 8  
somma(10, 20)      # 30
```

Restituire un risultato: `return`

Finora le funzioni hanno solo stampato qualcosa. Ma spesso vuoi che la funzione **calcoli qualcosa e ti restituisca il risultato**, in modo da poterlo usare dopo.

Si usa la parola chiave `return` :

```
def somma(a, b):  
    return a + b  
  
risultato = somma(3, 5)  
print(risultato)          # 8  
print(somma(10, 20) * 2) # 60 - puoi usarlo direttamente nelle espressioni
```

Quando Python incontra `return` , esce dalla funzione e consegna il valore. Se una funzione non ha `return` , restituisce `None` automaticamente.

Valori predefiniti per i parametri

Puoi assegnare un valore di default a un parametro. Se non viene passato, usa quel valore:

```
def saluta(nome, saluto="Ciao"):
    print(f"{saluto}, {nome}!")

saluta("Alice")          # Ciao, Alice! – usa il default
saluta("Bob", "Buongiorno") # Buongiorno, Bob! – sovrascrive il default
```

I parametri con valore di default devono sempre venire **dopo** quelli senza.

Passare argomenti per nome

Puoi specificare il nome di ogni parametro quando chiami la funzione. L'ordine diventa irrilevante:

```
def descrivi_persona(nome, eta, citta):
    print(f"{nome} ha {eta} anni e vive a {citta}.")

# Ordine normale
descrivi_persona("Alice", 16, "Roma")

# Con i nomi – l'ordine non importa più
descrivi_persona(eta=16, citta="Roma", nome="Alice")
```

Restituire più valori

Una funzione può restituire più di un valore contemporaneamente:

```
def min_max(numeri):
    return min(numeri), max(numeri)

piccolo, grande = min_max([3, 1, 4, 1, 5, 9])
print(piccolo)  # 1
print(grande)   # 9
```

Esempio pratico

```
def calcola_media(voti):
    if len(voti) == 0:
        return 0
    return sum(voti) / len(voti)

def classifica_voto(media):
    if media >= 9:
        return "Ottimo"
    elif media >= 7:
        return "Buono"
    elif media >= 6:
        return "Sufficiente"
    else:
        return "Insufficiente"

voti_alice = [8, 7, 9, 6, 10]
media = calcola_media(voti_alice)
giudizio = classifica_voto(media)

print(f"Media: {media:.1f} – {giudizio}")
# Media: 8.0 – Buono
```

Nota come il programma principale è facile da leggere: capisce cosa fa senza dover sapere come funziona ogni funzione internamente.